ResourceX – Asset Lifecycle Platform

Individual Contribution Report & System Documentation

Indian Institute of Information Technology, Sri City (IIIT Sri City) • FSD 2024–2028
Multi-Tenant Enterprise Resource & Asset Allocation Management System

J Prem Sahith
Roll No: S20240010089
Branch: updated-features
Date: October 2026

EXECUTIVE CONTRIBUTION OVERVIEW

As a core full-stack developer on the **ResourceX** engineering team, my primary contributions focused on building and operationalizing critical sub-systems across both frontend (React 19, Vite, Modular CSS) and backend (NestJS, TypeScript, RBAC): **(1) Requestor Activity & History Center** for asset return and maintenance transparency; **(2) Inventory Staff Return Inspection & Tri-State Routing Engine**; **(3) Registrar Procurement Approvals, Document Previews & Institutional Analytics**; and **(4) Marketing Landing Page Conversion with Scroll-Spy Architecture**. All modules have been developed, typed, tested, and pushed to remote branch `updated-features`.

KEY MODULES DEVELOPED & SPECIFIC FILES

1. Requestor History & Activity Center (Returns & Maintenance Tracking)

FRONTEND • BACKEND • FULL-STACK

Architected a dedicated, user-facing history portal providing equipment requestors with full transparency over returned assets, condition assessments, and repair service tickets.

Dual-Tab Navigation (Returns vs Maintenance) | Clean Badge: Return Accepted & Closed | Processed by Inventory Indicator | Condition & Status Filtering | Requestor Isolation Guard

- `react-frontend/src/components/dashboards/requestor/views/RequestorHistoryView.jsx`
  Engineered unified history view. Replaced raw internal warehouse logistics tags (e.g. *Sent to Maintenance*) with client-reassuring badge ✅ `Return Accepted & Closed` and `Processed by Inventory` subtext. Integrated search, condition filtering (*Good*, *Fair*, *Damaged*, *Bad*), and repair ticket status monitoring.
- `react-frontend/src/components/dashboards/requestor/RequestorDashboard.jsx`
  Integrated the new `History & Activity` navigation route and dynamic view switching.
- `react-frontend/src/components/dashboards/requestor/views/RequestorOverview.jsx`
  Added real-time *Returned Assets* KPI card and quick-launch activity center banner.
- `react-frontend/src/components/dashboards/requestor/views/MyResourcesView.jsx`
  Added contextual jump button linking directly to return and repair history records.
- `react-frontend/src/services/requestor.js`
  Built API client abstractions `fetchRequestorReturnHistory()` and `fetchRequestorMaintenanceHistory()`.
- `backend/src/returns/returns.service.ts` & `backend/src/maintenance/maintenance.service.ts`
  Implemented query scoping ensuring Requestors only retrieve their own return records and maintenance tickets (filtering by authenticated user identity).

2. Inventory Staff Return Management & Inspection Queue

FRONTEND • BACKEND ROUTING

Transferred equipment return inspection from maintenance staff to **Inventory Staff** to match realistic operational resource lifecycles.

Real-time Inspection Queue | Multi-condition Evaluation | Tri-state Dispatch (Available / Maintenance / Scrap) | Audit Return History Logs

- `react-frontend/src/components/dashboards/inventory-officer/views/InventoryReturnsView.jsx`
  Built dual-view return workstation: (a) *Inspection Queue* modal to assess physical condition and determine resource destination (Available / Maintenance Queue with issue notes / Scrapped); (b) *Return History Logs* with destination filtering and asset search.
- `react-frontend/src/components/dashboards/inventory-officer/InventoryOfficerDashboard.jsx`
  Configured `Process Returns` navigation link and sub-view routing.
- `react-frontend/src/components/dashboards/inventory-officer/views/InvOfficerOverview.jsx`
  Added pending return KPI counters and operational quick-action banner.
- `react-frontend/src/services/inventoryOfficer.js`
  Implemented `fetchReturnHistory()` and `processReturn()` API methods.
- `backend/src/returns/returns.controller.ts`
  Updated endpoint authorization guards with `@Roles('Inventory Officer', 'Maintenance Officer', 'System Admin')`.

3. Registrar Portal React Conversion & Global Analytics

REACT 19 • SPA ARCHITECTURE

Architected and converted the complete Registrar institutional procurement, file previewing, and analytics suite from static pages to an enterprise React SPA.

- `react-frontend/src/components/dashboards/registrar/RegistrarDashboard.jsx`
  Master dashboard shell synchronizing sidebar navigation, active tab states, and URL parameters.
- `react-frontend/src/components/dashboards/registrar/views/RegistrarOverview.jsx`
  High-level KPI dashboard with resilient parallel asynchronous data loading (`Promise.all`).
- `react-frontend/src/components/dashboards/registrar/views/ProcurementApprovalsView.jsx`
  Procurement review suite supporting one-click approval, cost summaries, and rejection with mandatory justification.
- `react-frontend/src/components/dashboards/registrar/views/GlobalRequestsView.jsx`
  Cross-departmental request audit table with dynamic backend department filtering.
- `react-frontend/src/components/dashboards/registrar/views/SystemAnalyticsView.jsx`
  Operational capacity analytics and department resource allocation metrics.
- `react-frontend/src/components/dashboards/registrar/components/FilePreviewModal.jsx`
  In-browser document viewer for specification sheets and vendor invoices (supports PDF iframe embedding, image zoom, Base64-to-Blob decoding, and local download).
- `react-frontend/src/components/dashboards/registrar/components/RegistrarToast.jsx` & `services/registrar.js`
  Floating toast feedback notification system and modular API client connecting to NestJS endpoints.

4. Marketing Landing Page React Conversion & Scroll-Spy

UI/UX • CSS DESIGN TOKENS

Converted static HTML marketing website into an animated, modular React component tree with automated viewport tracking.

- `react-frontend/src/components/landing/LandingPage.jsx` & Sub-components
  Built modular component architecture: `Navbar`, `HeroSection`, `DepartmentsSection`, `UsersSection`, `ResourcesSection`, `PartnersSection`, `Footer`.
- `react-frontend/src/components/landing/Navbar.jsx`
  Engineered automated **Scroll-Spy** using native `IntersectionObserver` to update active navbar state during scrolling without performance lag.
- `react-frontend/src/components/landing/landing.css`
  Scoped modern CSS design system with custom HSL palettes, smooth transitions, and mobile/tablet responsive breakpoints.

VERIFICATION, TESTING & BUILD VALIDATION

TARGET DOMAIN	TEST / VERIFICATION PERFORMED	COMMAND / TOOL	RESULT STATUS
Backend Type Safety	Strict NestJS TypeScript compilation across controllers, services, guards	`npx tsc --noEmit`	0 Errors / Passed
Frontend Bundle	Production asset bundling & JSX syntax verification via Vite	`npm run build`	147 Modules / Passed
Return Isolation	Verification that Requestors cannot view other users' return handovers	API User-context check	Scoped to Auth User
Status Badge Check	Verification that 'Sent to Maintenance' is replaced with clean badge	Component Test	Verified Clean
Git Versioning	Synchronized commits and branches pushed to origin repository	`git status & push`	Up to Date